

OKX Pulse: Real-Time HFT Signal Inference with Executable-PnL Labels

Tuan Minh Pham
Course Project Report
COSC6376 - Cloud Computing
Email: tmpham42@cougarnet.uh.edu

Abstract—This report describes OKX Pulse, a course project for real-time trading signal inference on OKX market data. The system uses a C++ runtime, a Python machine-learning pipeline, an Electron interface, and Docker/AWS deployment files. The main idea is to train and test signals with executable bid/ask profit-and-loss labels, not only future price direction. This target is harder because a trade must cover the spread before it can make profit. The system shows a local gRPC loop between the C++ runtime and the Python inference service. It also compares XGBoost and LightGBM on a dataset where most labels are neutral, and compares ECS on EC2 with ECS on Fargate. The final test results did not give positive executable PnL. However, the results show that order-book and trade-flow features still have useful information. More work is needed on labels, feature parity, and threshold choice.

Index Terms—high-frequency trading, market microstructure, order book, machine learning, gRPC, ECS, Fargate, OKX

I. INTRODUCTION

High-frequency trading systems are hard to evaluate with simple price-direction labels. A model can predict a small future mid-price move correctly, but the trade can still lose money after the spread, fees, latency, and execution limits. OKX Pulse handles this problem by using executable trading edge. The project does not only ask whether the future price goes up or down. It asks whether buying at the current ask and selling at a future bid, or selling at the current bid and buying back at a future ask, would give positive PnL.

The project has three goals. First, it builds a real-time service architecture for OKX market data and gRPC streaming [1], [2]. Second, it keeps online C++ features aligned with offline Python training features. This helps reduce train/serve drift. Third, it prepares local container and AWS deployment files, then compares ECS on EC2 with ECS on Fargate.

II. SYSTEM ARCHITECTURE

The system has two main paths: online runtime and offline training. The online path is latency-sensitive, so it stays local and uses C++ for market data, feature building, strategy logic, and paper trading [3]. Python is used for model serving and offline machine-learning work.

A. Online Runtime

The online runtime starts from OKX WebSocket data. The C++ backend reads book and trade updates, keeps rolling feature state, can write feature rows to CSV, and publishes a

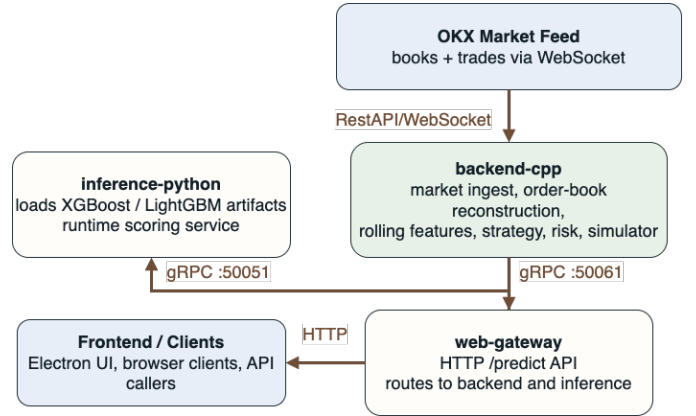


Fig. 1. Runtime architecture. The C++ service reads OKX market data, builds features, streams data over gRPC, and calls the Python inference service.

TABLE I
RUNTIME SERVICES.

Service	Stack	Role
backend-cpp	C++	OKX ingest, feature generation, strategy, paper trading, and gRPC streaming
inference-python	Python	Model loading and prediction service
web-gateway	Node.js	HTTP routing for clients and UI access

MarketData gRPC stream on port 50051. The Python inference service exposes a PredictionService gRPC endpoint on port 50061. Runtime messages can also include prediction fields such as model name, signal, return estimate, price estimate, and detail text.

B. Offline Training Path

The offline path converts captured or historical market data into feature rows. It then builds labels from future executable PnL, trains models, tests thresholded trading results, and stores model files. This separation makes experiments easier while keeping a clear link between training columns and online feature names.

III. DATA CONTRACTS

The repository uses two main data contracts. The live service contract is defined in

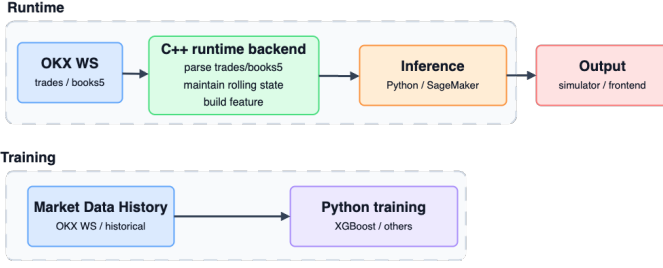


Fig. 2. Feature and label-building flow. Book and trade data are converted into features, then future bid/ask values are used to build executable-PnL labels.

backend/proto/market_data.proto. It includes a streaming MarketData.Subscribe interface and a request/response PredictionService.Predict interface. The offline contract is a normalized CSV feature table. The runtime creates this table, and the Python training pipeline uses it.

The most important rule is feature parity. Online C++ feature names and offline Python training columns must stay aligned. If they do not match, train/serve drift can happen even when the model and service code run correctly.

IV. FEATURE ENGINEERING AND LABELS

The project uses market microstructure features in four groups: price state, order-book pressure, trade flow, and short-horizon changes. Price-state features include mid price, spread, relative spread, and microprice. Order-book features include level-1 and five-level imbalance. Trade-flow features include recent trade volume, trade imbalance, and trade VWAP. Short-horizon features track recent changes in mid price, spread, and book imbalance.

A. Executable-PnL Label

For a prediction horizon H , the project checks whether a crossed-spread trade would make profit. A long trade buys at the current ask and exits at the future bid:

$$\text{buy_pnl}_H = \text{bid}_H - \text{ask}_{\text{now}}. \quad (1)$$

A short trade sells at the current bid and exits at the future ask:

$$\text{sell_pnl}_H = \text{bid}_{\text{now}} - \text{ask}_H. \quad (2)$$

The class label is:

$$\text{label}_H = \begin{cases} 1, & \text{buy_pnl}_H > \epsilon \text{ and} \\ & \text{buy_pnl}_H > \text{sell_pnl}_H \\ -1, & \text{sell_pnl}_H > \epsilon \text{ and} \\ & \text{sell_pnl}_H > \text{buy_pnl}_H \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

Here, label 1 means executable long edge. Label -1 means executable short edge. Label 0 means no clear edge. This target is stricter than raw direction classification because it includes the spread cost.

TABLE II
DATASET SNAPSHOT.

Metric	Value
Rows	86,399
Columns	43
Train rows	69,119
Test rows	17,280
Split	chronological 80/20

TABLE III
EXECUTABLE-LABEL DISTRIBUTION.

Label	Train	Test
Down	6,341	2,159
Neutral	56,176	13,226
Up	6,602	1,895

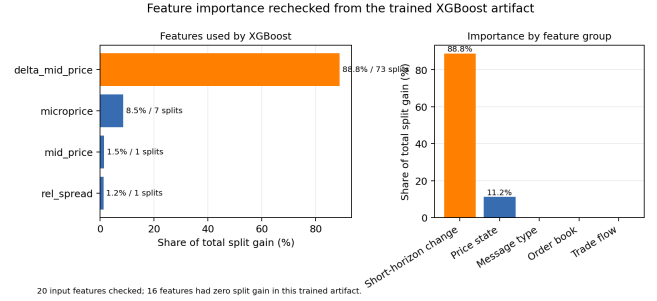


Fig. 3. Feature importance rechecked from the trained XGBoost artifact. Most split gain came from delta_mid_price and price-state features in this run.

V. MODELING RESULTS

The ML pipeline was rechecked from the saved dataset and evaluation files. The final comparison uses 86,399 rows and 43 columns. The data is split by time into 69,119 training rows and 17,280 testing rows. The training part is also split into 55,296 fitting rows and 13,823 calibration rows. The model uses 20 feature columns and predicts signed executable return. Most labels are neutral. This is expected because the label requires executable edge, not only a small price move.

The archive builder requested several days of OKX data, but the effective timestamps in this run came from April 12, 2026. Because of this, the result should be read as a one-day experiment. More days are still needed before the model can be called robust.

The feature-importance check gives an important warning. The trained XGBoost model does not use all feature groups equally. In this run, 16 of the 20 input features have zero split gain. Most useful signal comes from short-horizon mid-price change and price-state features. This means the current dataset is still too small or too narrow for many depth and trade-flow features to help much.

The final held-out comparison uses XGBoost and LightGBM [4], [5]. The two models have similar error size, but XGBoost has higher correlation with the signed executable return. Table IV shows the model quality on the test split. MAE and RMSE are shown in basis points.

TABLE IV
RETURN PREDICTION QUALITY ON THE TEST SPLIT.

Model	MAE	RMSE	Corr.	Dir. Acc.
XGBoost	0.311	0.670	0.233	23.47%
LightGBM	0.318	0.666	0.205	23.48%

TABLE V
HELD-OUT EXECUTABLE-PNL MODEL RESULTS.

Model	Th.	Trades	Cov.	PnL/BTC	PnL/\$5k	Win
XGBoost	0.3 bps	691	3.999%	-295.6	-20.76	29.23%
LightGBM	0.5 bps	248	1.435%	-604.7	-42.53	23.79%

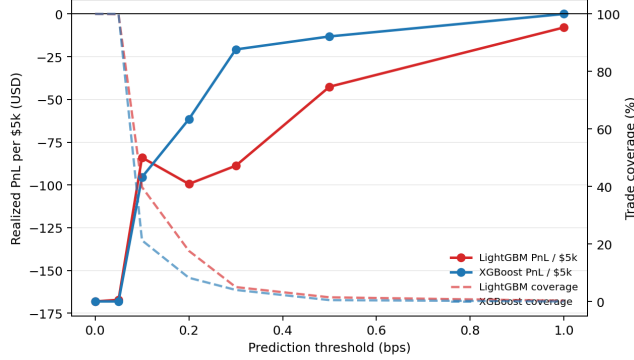


Fig. 4. Threshold sweep on the held-out test split. Higher thresholds reduce trade coverage and reduce the loss, but they do not make the strategy profitable.

Neither model reaches positive executable PnL on the test split after threshold selection. XGBoost makes 691 trades at a 0.3 bps threshold with -295.6 PnL per BTC and a 29.23% win rate. LightGBM makes 248 trades at a 0.5 bps threshold with -604.7 PnL per BTC and a 23.79% win rate.

The threshold sweep gives more useful information than one final number. With no threshold, both models trade almost every row and lose about \$168 per \$5k notional. A higher threshold reduces the number of trades and lowers drawdown. For XGBoost, the selected 0.3 bps threshold gives 4.00% coverage and about \$33.40 maximum drawdown per \$5k. For LightGBM, the selected 0.5 bps threshold gives 1.44% coverage and about \$47.50 maximum drawdown per \$5k.

These results are still useful. They show that the feature set has some structure, but the current signal is not strong enough to beat spread and execution costs. The expected PnL from the model is positive, but the realized PnL is negative. This means calibration is not reliable enough yet. The safest runtime choice is shadow mode, where predictions are logged and watched but not used for live trading.

VI. DEPLOYMENT EVALUATION

The deployment files package three services: backend-cpp, inference-python, and web-gateway. The local container runtime uses Docker Compose. The AWS files include ECS task definitions, ECS service templates, EC2 scripts, and Terraform notes [6].

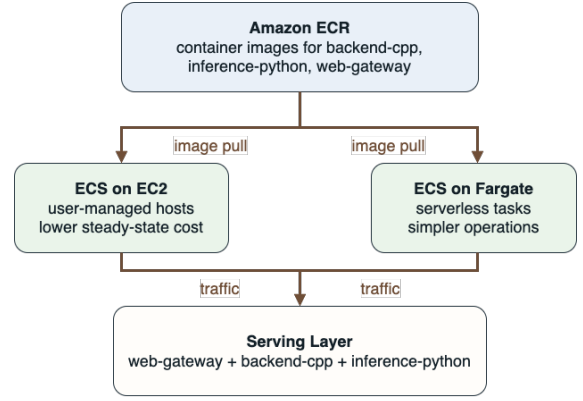


Fig. 5. Container deployment view for the runtime services.

The recommended deployment is ECS on EC2 for always-on serving. It gives lower steady-state cost and more host control. Fargate is still useful when simple operation matters more than cost. The project estimate gives about \$36.79 per month for three t4g.small EC2 nodes. The same three services on Fargate cost about \$90.10 per month, plus about \$1.20 per month for 11.9598 GiB of ECR storage.

VII. LESSONS LEARNED

The most important lesson is that label design has a large effect on trading results. Direction classification is easier, but executable PnL labels match the real decision better. The second lesson is that train/serve feature parity must be treated as a system contract, not only as a notebook detail. The third lesson is that deployment should match runtime needs. Cloud infrastructure is useful for packaging, service management, observability, and reproducibility. However, the live inference hop should stay local and internal for the main trading loop.

VIII. CONCLUSION

OKX Pulse shows a real-time HFT signal inference platform with C++ market-data processing, Python model serving, gRPC contracts, Electron UI support, and AWS container deployment files. The final models do not produce positive held-out executable PnL. However, the system gives a clear base for more work on data quality, feature parity, thresholding, and model choice. ECS on EC2 is the better option for cost-efficient always-on deployment. Fargate is better when lower operation work is more important. Executable-PnL labels should continue to be used for realistic signal testing.

REFERENCES

- [1] OKX, "API Documentation v5 and Historical Data." [Online]. Available: API docs; historical data. Accessed May 7, 2026.
- [2] gRPC Authors, "Documentation." [Online]. Available: grpc.io/docs. Accessed May 7, 2026.
- [3] VinaDevs, "BinanceQuantTrader." [Online]. Available: GitHub repo. Accessed May 7, 2026.
- [4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [5] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *NeurIPS*, vol. 30, 2017. [Online]. Available: NeurIPS page.
- [6] AWS, "ECS and Fargate Documentation." [Online]. Available: ECS docs; Fargate docs. Accessed May 7, 2026.